

Received 21 December 2024, accepted 6 March 2025, date of publication 13 March 2025, date of current version 25 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3551088

RESEARCH ARTICLE

Emperor Yu Tames the Flood: Water Surface Garbage Cleaning Robot Using Improved A* Algorithm in Dynamic Environments

RONGHAO LI¹, BINGYING ZHANG¹, DI LIN¹, RONG-GUEI TSAI², WENWEN ZOU³, SHANNA HE⁴, ZIQI LIN³, XINXIN CHEN¹, AND YATING YAO⁵

¹Putian Electronic Information Industry Technology Research Institute, New Engineering Industry College, Putian University, Putian, Fujian 351100, China

²College of Computer Science, Guangdong University of Science and Technology, Dongguan, Guangdong 523083, China

³School of Information Technology, Xiamen Huatian International Vocational Institute, Xiamen, Fujian 361000, China

⁴School of Mechanical and Electrical Engineering and Information, Zhangzhou Institute of Science and Technology, Zhangzhou, Fujian 363000, China

⁵Department of Digital Media Art, Fuzhou Technology and Business University, Fuzhou, Fujian 350715, China

Corresponding author: Di Lin (lindi1015@163.com)

This work was supported by the Innovation and Entrepreneurship Project under Grant X202411498028. The work of Di Lin was supported in part by Putian City Science and Technology Project, China, under Grant 2022GZ2001ptxy14 and Grant 2023NJJ008; and in part by Putian Electronic Information Industry Research Institute under Grant 2023GJGZ003. The work of Rong-Guei Tsai was supported in part by Putian City Science and Technology Project, China, under Grant 2022GZ2001ptxy14 and Grant 2023NJJ008; in part by Putian Electronic Information Industry Research Institute under Grant 2023GJGZ003; and in part by the Science Foundation of Fujian Province, China, under Grant 2020J01924, Grant 2023J011015, and Grant 2021J05243.

ABSTRACT Every year, the global production of marine debris reaches a staggering 400 million tons, and the amount of garbage continues to increase. Among this, about 14 million tons will eventually flow into the oceans, posing a significant threat to marine ecosystems. Apart from endangering marine ecology, this waste will ultimately come back to affect humans. To address the increasingly severe issue of marine debris, our team independently developed a surface garbage cleaning robot, DaYu No. 1, and designed a path planning algorithm specifically for DaYu No. 1, called DyNav. DaYu No. 1 aims to increase garbage cleaning efficiency and adaptability. The traditional A* algorithm performs well in static environments, but the ocean is a dynamic environment where marine debris constantly shifts due to ocean currents or tides, making it extremely challenging for robots to collect marine debris. Therefore, our proposed DyNav algorithm can adaptively adjust paths to cope with environmental changes. We proposed DyNav, a new path planning algorithm that takes ocean currents into account, and the DaYu No. 1 robot with YOLOv7 and IMA algorithms built in. These will make cleaning up marine debris much more efficient and flexible. Compared to traditional path planning algorithms, DyNav demonstrates superior performance. We have validated the effectiveness of the DyNav algorithm through practical experiments. The results show that in dynamic environments, the robot can clean surface debris more quickly and flexibly while avoiding obstacles. This not only contributes to environmental protection, but also improves cleaning efficiency, offering an innovative solution for water environment management.

INDEX TERMS Water surface garbage cleaning robots, A* algorithm, path planning, Greedy, RRT.

I. INTRODUCTION

Every year, the global production of marine debris reaches a staggering 400 million tons, and the amount of waste

The associate editor coordinating the review of this manuscript and approving it for publication was Tao Liu¹.

continues to escalate. As a result, around 14 million tons eventually find their way into the oceans, posing a significant threat to marine ecosystems. Estimates suggest that by 2050, the weight of plastic waste in the oceans will surpass that of fish, thereby intensifying the severity of the marine debris issue [1]. The presence of marine debris not only reduces

marine biodiversity and endangers many species, but it also affects food security by transferring contaminants along the food chain. Apart from jeopardizing marine ecosystems, marine debris' repercussions ultimately circle back to impact human health.

Currently, approximately 80% of marine debris originates from land, including waste from rivers, sewers, landfills, and other terrestrial sources, while the remaining 20% comes from marine activities such as fisheries, shipping, and maritime operations [2]. In terms of marine debris cleanup, litter on beaches can be removed through beach cleaning activities, but debris floating on the water surface or deposited on the seabed is often left to drift due to the difficulty of cleaning. Fortunately, with advancements in robotics engineering, the Internet of Things, communication technologies, and marine engineering, utilizing robots to assist in marine debris cleanup has become a viable solution. These robots can adapt to the dynamic marine environment, possess obstacle avoidance capabilities, and can save time and labor costs. Importantly, employing robots for marine debris cleanup can mitigate the dangers associated with manual cleanup efforts, and robots offer advantages such as sustainability and long-duration operation.

Based on the aforementioned reasons, our team has independently developed a surface debris cleaning robot named DaYu No. 1. In ancient Chinese legend, Emperor Yu devoted 13 years to developing systematic flood control strategies through persistent exploration and adaptive governance, ultimately protecting communities from water-related disasters. This narrative resonates deeply with our robot's mission: to continuously "tame" water pollution through intelligent waste collection, mirroring Yu's spirit of sustained environmental stewardship and harmonious coexistence with water ecosystems. Therefore, we have named the robot **DaYu No. 1**. In the technical research of surface debris cleaning robots, path planning is one of the core aspects and a critical technology for achieving autonomous navigation [3]. Numerous studies currently focus on solving the path planning issue for surface robots using various algorithms. However, these methods are all based on the assumption that the target objects and marine environment are static. In reality, factors like ocean currents and wind cause the position of marine debris to constantly change, adding complexity to robot path planning. Therefore, designing a dynamic path planning algorithm that can adapt to the marine environment is more practical. The continuous changes in marine debris location information make static path planning methods inapplicable, increasing the difficulty of path planning. In this study, we have designed a path planning algorithm named DyNav for DaYu No. 1, aiming to enhance the efficiency and adaptability of debris cleaning. Due to the dynamic nature of the ocean, marine debris constantly shifts position due to influences like ocean currents or tides, making it challenging for robots to retrieve marine debris. Therefore, our proposed DyNav can dynamically adjust paths to cope with environmental changes.

Furthermore, to achieve the goal of saving time and effort, we aim to find the shortest path that enables the surface cleaning robot to traverse each piece of marine debris exactly once. We can model this problem as the Traveling Salesman Problem (TSP), an NP-hard problem, due to the robot's excellent mobility and ability to collect all marine debris [4].

Figure 1 shows the surface debris cleaning robot: DyNav No. 1. To achieve the goal of continuous operation for the surface debris cleaning robot, this study aims to find the shortest path. The robot starts from the initial point, traverses all marine debris, and then returns to the charging station (starting point) for recharging to enable continuous operation. The objective is to find the shortest path to save time and electrical costs. Therefore, we can model this problem as solving the Minimum Hamiltonian Cycle problem, which is an NP-hard variation of the Traveling Salesman Problem. For these reasons, constructing a path planning solution suitable for dynamic marine environments has become a challenging task.



FIGURE 1. The surface debris cleaning robot: DyNav No. 1.

This study proposes a heuristic adaptive path planning algorithm named DyNav based on the A* algorithm. Compared to traditional path-planning algorithms, DyNav demonstrates superior performance. We have validated the effectiveness of the DyNav algorithm through practical experiments. The results indicate that in dynamic environments, the robot can swiftly and flexibly clean surface debris while avoiding obstacles. This not only contributes to environmental protection, but also enhances cleaning efficiency, improves marine debris retrieval, reduces labor and time costs, and offers an innovative solution for water environment management.

This study makes contributions in three main aspects:

1) Based on the A* algorithm, this study has developed a heuristic adaptive path planning algorithm named DyNav. DyNav leverages the strengths of the A* algorithm, known for its ease of implementation and efficiency. The study enhances the heuristic function and crafts a path planning solution

tailored for dynamic settings, boosting planning efficiency and adaptability to environmental shifts. Compared to algorithms like Greedy, RRT, SMURF, and U*, DyNav excels in marine debris cleanup volume and reduction in the robot's total path length.

2) Our research team independently created a surface debris cleaning robot, DaYu No. 1, integrating YOLOv7 and IMA depth localization technologies. This innovation significantly bolsters technological support for marine environmental preservation and cleanliness.

3) The heuristic function of DyNav factors in ocean currents' speed and direction, reducing power consumption, enhancing debris traversal speed, and thereby improving floating debris cleaning efficiency. These holistic enhancements offer robust technical backing for future marine environmental conservation endeavors and serve as valuable benchmarks for similar initiatives.

The remaining sections of this paper are structured as follows: Section II describes existing solutions for marine debris removal and path planning algorithms. Section III defines marine debris traversing as a dynamic TSP and elaborates on the DyNav algorithm's detailed process. Section IV presents simulation experiments and field tests of the DaYu No. 1 robot, testing DyNav, Greedy, RRT, SMURF, and U*. The final section includes the conclusions of this study and outlines future work.

II. RELATED WORK

The current solutions for addressing marine debris issues mainly fall into two categories: surface garbage cleaning robots [5], [6], [7], [8], [9], [10], [11], [12], [13] and underwater garbage bins [14], [15], [16]. Surface garbage cleaning robots have the advantage of being mobile, but they come with higher costs and implementation challenges. On the other hand, underwater garbage bins are cost-effective and simple to install, but they lack mobility. With their mobility, surface garbage cleaning robots can save energy and improve the efficiency of marine garbage collection, but they must take into account the crucial issue of path planning [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37].

A. PATH PLANNING ALGORITHMS FOR A SURFACE GARBAGE CLEANING ROBOT

To achieve the goal of comprehensive water surface cleaning, in [17], Zhu et al. designed a fully autonomous water surface cleaning robot, SMURF. They introduced a novel water surface coverage path planning method and an improved nonlinear model predictive controller. In [18], Xu et al. presented the Completely Covered Neural Network (CCNN) algorithm for Unmanned Surface Vehicle (USV) path planning for the first time. The CCNN algorithm significantly reduces computational time by simplifying neural activity calculations.

Traditional A* algorithms have limitations in path planning, making them unsuitable for underwater unmanned vehicle movement. To address these drawbacks, Wang et al.

introduced an improved A* algorithm in [19] that smooths and optimizes autonomous underwater vehicle (AUV) paths through path optimization. In [20], Lin et al. proposed an enhanced A* algorithm that incorporates extra safety margins to filter impassable narrow passages and prevent collisions between vehicles without obstacles.

Biomimetic algorithms are also commonly used for path planning. Their advantages include strong global search capability, adaptability to different environments and types of problems, and the ability to handle nonlinear problems. However, the drawbacks of biomimetic algorithms include their slow convergence speed, challenges in parameter settings that significantly impact performance, and the risk of becoming trapped in local optimal solutions. In [21], Su et al. formulated constraints and objective functions for global path planning of autonomous underwater vehicles (AUVs), taking into account factors such as terrain threats, navigation distance, and energy loss. They introduced the GA-SQP algorithm by combining genetic algorithms (GA) and sequential quadratic programming (SQP) to achieve global path planning for AUVs in complex underwater environments. In [22], Patle et al. addressed navigation challenges by employing a hybrid approach using the Firefly algorithm and the Genetic algorithm. This method effectively processes sensory information to make accurate decisions, resolving obstacle avoidance and target seeking in navigation. In [23], Zhong et al. proposed a strategy of increasing the number of offspring using multi-domain inversion to overcome inherent flaws in traditional genetic algorithms. By conducting a second adaptive evaluation to eliminate poor offspring and retain the optimal individuals, this improvement enhances local search capabilities and increases the probability of generating excellent individuals. In [24], Tao et al. established an AUV path planning model based on an improved genetic algorithm. The model primarily focuses on minimizing path length while taking into account collision avoidance ranges and turning angle constraints. They designed a reasonable adaptive probability model for crossover and mutation. In [25], Bresciani et al. introduced a Genetic Path Planner (GPP) based on the Informed Path Planning (IPP) strategy using genetic algorithms. This planner aims to generate paths that maximize both information collection and area coverage simultaneously.

In [26], Hu et al. proposed a hybrid algorithm combining Ant Colony Optimization (ACO) and Genetic Algorithm (GA) for autonomous underwater vehicle path planning. This novel algorithm merges the characteristics of both ACO and GA. Wang et al. [28] described a new hybrid algorithm (IPSO-GA) for planning the best path for UAVs to collect data. It is a combination of an improved particle swarm optimization algorithm and a genetic algorithm. The IPSO-GA hybrid algorithm, which is a combination of the improved particle swarm optimization algorithm and the genetic algorithm, makes a path that covers all data nodes as efficiently as possible. In [29], Huang et al. presented the APSO algorithm, which combines A* and particle swarm

optimization (PSO) for calculating the optimal path. APSO adds random inertia weights and adversarial learning strategies to make PSO work better by improving the search capabilities and making the A* algorithm better by getting rid of unnecessary points.

The RRT (Rapidly exploring Random Tree) algorithm features rapid exploration and randomness. It exhibits excellent scalability and adaptability in high-dimensional spaces and can effectively handle complex environments and dynamic obstacles. The RRT algorithm is typically able to rapidly find feasible paths and performs well in dynamic environments. The search space may limit the optimal paths generated by the RRT algorithm. In [30], Du et al. proposed an Asymptotically Optimal Rapidly-exploring Random Tree (RRT) method for mobile robot path planning in dynamic environments. They introduced a heuristic sampling algorithm to improve sampling efficiency and optimized paths through post-processing procedures, taking into account path length and safety. In [31], Khattab et al. introduced a new variant of the RRT* algorithm aimed at improving path generation efficiency, including computation time and path quality. In [32], Lin et al. proposed a reinforcement learning-based RRT algorithm for pathfinding in unfamiliar environments. In [33], Shao et al. introduced a real-time path planning method using the Dynamic Enhanced Fireworks Algorithm-APF for automobiles. By considering vehicle dynamics and various obstacles, this method seeks to enhance path safety and smoothness, aiming to find the most cost-effective path in driving areas. In [34], Xiangde et al. presented a dynamic path planning approach that integrates global and local planning. They are using a Harmony Search algorithm. They achieve optimization through adversarial and optimal learning, adjusting dynamics and memory retrieval rates in a harmonious manner. They develop a global dynamic path planning algorithm that considers global path optimization and real-time obstacle avoidance.

Reinforcement learning excels in decision-making sequences within complex environments. To introduce priority weight into the Q-learning algorithm to enhance its value estimation in practical scenarios. Dynamic obstacle avoidance path planning applies this improved algorithm [35]. In [36], Bai et al. proposed an enhanced deep reinforcement learning path planning algorithm for optimizing autonomous mobile robots (AMRs) paths. They model the path planning for AMRs as a Markov Decision Process and use a Double Deep Q Network to obtain optimal solutions. In [36], Vashisth et al. advocate for a novel deep reinforcement learning approach for adaptive replanning of robot paths to locate targets in unknown 3D environments.

Local optimal solutions constrain the A* algorithm, which guarantees the shortest path but exhibits high computational complexity in vast environments. Hybrid algorithms amalgamate the strengths of different methods but involve higher implementation complexity. Implementing the artificial potential field algorithm is simple, making it suitable for obstacle avoidance and target guidance. However,

TABLE 1. A comparison of common path planning algorithms and their characteristics is presented.

Algorithm	Advantage	Disadvantage
Heuristic Algorithm [17], [18]	(1) Ability to quickly find solutions (2) The system is suitable for path planning in complex environments	(1) suitability of the heuristic function could potentially restrict it (2) necessitates adjusting parameters to adapt to various situations
A* based [19], [20]	(1) Ability to guarantee the shortest path (2) Ideal for static environments	(1) High computational complexity occurs in enormous environments. (2) Limited by local optimal solutions
Hybrid [27], [29], [32], [33]	(1) Combines the strengths of different algorithms, considering multiple factors comprehensively	(1) To ensure optimal performance, parameters must be fine-tuned. (2) Implementation complexity is higher
Artificial Potential Field [34]	(1) Simple and simple to implement (2) The device is suitable for obstacle avoidance and target guidance	(1) May get stuck in local optimal solutions (2) Requires appropriate adjustment of the potential field function to achieve optimal results
Biomimetic Algorithm (GA, PSO, Ant etc.) [21]-[26], [28]	(1) Possess global search capability and strong adaptability	(1) Requires a large amount of parameter tuning (2) Convergence speed may be slower
RRT based [30], [31]	(1) Suitable for rapid exploration in complex environments	(1) Cannot guarantee finding the optimal solution (2) May require a long time to converge
Reinforcement Learning [35]-[37]	(1) Capable of learning and optimizing paths in unknown environments	(1) Requires a large amount of training data (2) May have a slower convergence speed
DyNav (Proposed)	(1) Applicable to dynamic environments (2) Combines the advantages of A*, providing guarantees for the shortest path	(1) High computational complexity in very large environments (2) Limited by local optimal solutions

it may become stuck in local optimal solutions, requiring proper adjustment of the potential field function for optimal outcomes. Biomimetic algorithms offer global search capabilities and adaptability but require extensive parameter tuning, with potentially slower convergence speeds. RRT algorithms excel in quick exploration of complex environments but do not ensure the optimal solution and may require extended convergence times. Reinforcement learning is adept at learning and optimizing paths in unknown environments but demands abundant training data and may exhibit slower convergence rates.

Given the limitations of existing algorithms, DyNav emerges. DyNav is applicable in dynamic environments,

harnessing the advantages of the A* algorithm to ensure the shortest path. However, challenges persist in huge environments, including high computational complexity and constraints from local optimal solutions. Thus, the introduction of DyNav aims to address the deficiencies of current algorithms, providing a more effective and comprehensive solution for path planning issues in dynamic environments.

III. SYSTEM ARCHITECTURE AND PROBLEM DESCRIPTION

A. PROBLEM DESCRIPTION AND MODELING

Before explaining the DyNav algorithm, it is necessary to first define and describe the problem. Considering Figure 2, we deploy a DaYu No. 1 robot in the environment, equipped with a GPS receiver for accurate self-positioning. Additionally, we equip the robot with a laser radar and a camera, utilizing YOLOv7 [39], [40], [41] and Image Measurement Approach (IMA) positioning technologies [42] to effectively detect and locate marine debris and obstacles within a 5-meter range.

To effectively position objects, marine debris beyond the 5-meter detection range is considered invisible. The DaYu No. 1 robot is equipped with ocean current sensors that detect the speed and direction of the currents. As mentioned in [3], over 80% of marine debris is located at or within one meter below the water surface. Therefore, this study focuses on cleaning marine debris at or below the water surface. This study will set up a charging station to allow the robot to return and recharge after completing its task. This problem can be modeled as a Hamiltonian cycle problem.

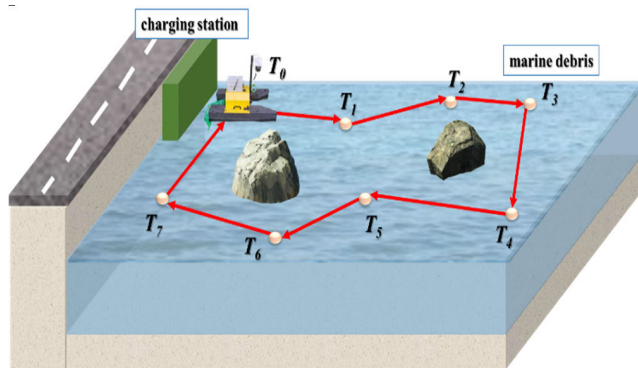


FIGURE 2. An example of the garbage-cleaning mission.

We can transform the problem into solving a Hamiltonian cycle, an NP-hard problem [4], to enable the robot to traverse the entire area of marine debris once and only once, returning to the starting point. Assuming there are n marine debris items in the environment, they can be represented as $\{T_0, T_1, \dots, T_{n-2}, T_{n-1}\}$. Starting from the point T_0 , the robot traverses each marine debris item once and finally returns to T_0 (Hamiltonian cycle). Therefore, we can represent the robot's trajectory $\mathbf{Traj}$ as equation (1):

$$\mathbf{Traj} = \{T_0, T_1, T_2, \dots, T_{n-1}, G_n, T_0\}. \quad (1)$$

The calculation of the total path length, denoted by $\varepsilon(\mathbf{Traj})$, is determined as shown in equation (2). Here, $\varepsilon(i)$ and $\varepsilon(j)$ represent any two points i and j within the trajectory $\mathbf{Traj}$.

$$\varepsilon(\mathbf{Traj}) = \sum_{i=1}^n \sum_{j=1}^n d(i, j) \eta_{ij}. \quad (2)$$

where Eq. (2) is limited by Eq. (3),

$$\begin{aligned} \sum_{i=1}^n \eta_{ij} &= 1, \quad \text{where } j = 1, 2, \dots, n-1 \\ \sum_{j=1}^n \eta_{ij} &= 1, \quad \text{where } i = 1, 2, \dots, n-1 \\ \eta_{ij} &= \{0, 1\}, \quad \text{where } i, j = 1, 2, \dots, n-1 \end{aligned} \quad (3)$$

The variable η_{ij} here is a binary variable; if there exists an edge between points i and j , then $\eta_{ij} = 1$; otherwise, $\eta_{ij} = 0$.

To minimize time and power costs, we aim for the shortest path, which is a minimum cycle. Therefore, we can obtain the shortest path, γ , through equation (4).

$$\gamma = \arg \min \varepsilon(\mathbf{Traj}). \quad (4)$$

B. THE STRUCTURE AND DESIGN OF DaYu NO. 1

Our team independently developed DaYu No. 1, a surface debris cleaning robot, specifically for environmental protection and marine cleanliness. This robot combines advanced technology and innovative design to efficiently clean up debris on the water surface, fulfilling a crucial mission in environmental and marine conservation.

DaYu No. 1 is a robot measuring 120 cm in length, 74 cm in width, and 130 cm in height, with a carrying capacity of at least 6.0 kg. The BeiDou navigation satellite system equips it, ensuring positioning accuracy within a range of approximately 1.2 meters. With a powerful propulsion force of 21.0 kg, it has a communication range of up to 1.7 kilometers. The Raspberry Pi 4 Model B serves as the development board, boasting an ARM Cortex-A72 1.5GHz 64-bit quad-core processor and 8GB RAM. DaYu No. 1 is a programmable robot, allowing for the integration of various path planning algorithms such as DyNav, Greedy, RRT, SMURF, and U* for performance validation to achieve efficient task planning. Table 2 details the hardware specifications.

DaYu No. 1 has advanced object detection and localization technologies for target detection and localization. By employing YOLOv7 [39], [40], [41] and IMA [42] techniques, it can swiftly and accurately identify target objects (marine debris) and obstacles, locate these objects precisely, and effectively guide the robot in carrying out cleaning operations.

DaYu No. 1 is a powerful and efficient surface debris cleaning robot that combines advanced technology and innovative design to contribute to environmental conservation efforts and showcase the team's exceptional capabilities in robotics. This robot can effectively detect and clean debris of various sizes, including plastics, paper, and other floating materials. Equipped with advanced sensors and obstacle avoidance

technology, the robot can intelligently navigate around obstacles, ensuring smooth cleaning operations while minimizing the risk of damage. The BeiDou navigation satellite system's integration with advanced positioning technology enables autonomous navigation, precise position control, and efficient completion of designated cleaning paths.

DaYu No. 1's versatility and potential value in various applications extend beyond surface debris cleaning to other fields such as scientific research surveys, water quality monitoring, or rescue missions.

TABLE 2. Hardware specifications of Dayu No. 1.

Item	Specification
Size of robot	Length: 120cm (47.2 in)
	Width: 74cm (29.1 in)
	Height: 130cm (51.2 in)
load-carrying capacity	6.0 kg
Localization error	BeiDou Navigation Satellite System (BDS) / $\pm 1.2m$
propulsion power	21.0 kg
Communication range	1.7 km
development board	Raspberry Pi4Model B
CPU	ARM Cortex-A72 1.5GHz
RAM	64-bit quad-core
	8 GB
Path planning algorithms	DyNav, Greedy, RRT, SMURF, U*
Object detection and localization	YOLOv7 [39]-[41] + IMA [42]

IV. PROPOSED PATH PLANNING ALGORITHM: DyNav

The DaYu No. 1 robot uses YOLOv7 and IMA technologies to determine the positions of all marine debris items. We enclose obstacles in a rectangular box, as shown in Figure 1. Therefore, after obtaining the coordinates of the four corners of all obstacles and the positions of the marine debris, we establish an undirected graph. This graph can be transformed into a distance matrix $\mathbf{DMAT} \in R^{((M+1) \times (M+1))}$, where $\mathbf{DMAT}$ is defined as the Euclidean distance between any two pairs of points. The diagonal distances of matrix $\mathbf{DMAT}$ are all 0, as shown in equations (5) and (6). Furthermore, as the robot traverses a piece of marine debris, it detects and updates the marine debris's position information simultaneously.

$$\mathbf{DMAT} = \mathbf{DMAT}^T. \quad (5)$$

$$\text{Diag}(\mathbf{DMAT}) = 0. \quad (6)$$

A. DISCOVERING TARGETS WITH THE OBJECT DETECTION ALGORITHM (YOLOv7)

The robot must first identify the target location before executing a task. In this study, we refer to the grid where each marine debris item is located, assuming there are N items in the environment, as the target location. This research utilized the

YOLOv7 (You Only Look Once version 7) object detection algorithm [39], [40], [41], [43]. However, in our application, we focus solely on detecting the presence of objects without further object identification. Despite the initial design of YOLOv7 to simultaneously detect and classify objects in images, we modified the algorithm to retain only the presence information of detected objects, eliminating the need for additional object classification.

Such a choice brings several significant advantages. Firstly, focusing solely on detecting the presence of objects speeds up the algorithm. By eliminating the need for complex object recognition and classification, we can detect objects in images more quickly, which is particularly crucial for applications requiring real-time feedback or extensive image processing. Furthermore, this modification makes the algorithm more lightweight, reducing computational costs and resource consumption while maintaining efficient object detection capabilities. We modified the YOLOv7 algorithm by adjusting its output layer, retaining only the object's presence information without including further classification. We adjust the loss function and post-processing steps in this modification to obtain only binary presence information of objects.

The requirements of our research scenario, which emphasizes the need for quick determination of object presence rather than further object categorization, served as the main inspiration for the technical choice. Additionally, we set a detection range and traversed all detected target locations for marine debris. To prioritize exploring distant marine debris during traversal and avoid redundant paths, the distance between the traversal target locations and the robot's position must be less than or equal to thd . Assuming the first traversed target location is G_i and the next is G_{i+1} , the distance between the two target locations must be less than or equal to thd . This design prevents the robot from first traversing distant target locations and then closer ones, thereby minimizing redundant paths. Furthermore, to ensure accurate detection of target objects by YOLOv7, thd was set to a range of 5 meters in this study. We set the distance at 5 meters for two reasons: first, it filters out distant targets to prevent the robot from planning paths for them during the candidate sequence generation process, which could lead to longer robot trajectories. Secondly, we utilize our previously proposed IMA depth localization technology [42] (in Part B). Studies have shown that images captured by the camera at a distance of 30 meters from the target object introduce a 1-meter error. To ensure the accuracy of depth localization, we adopt a 5-meter distance with the aim of keeping the localization error below 0.1 meters.

$$G_i = \text{BFS}(\mathbf{DMAT}) \\ s.t. \text{dist}(G_i, G_{i+1}) \leq thd \quad (7)$$

B. DEPTH LOCALIZATION OF TARGET OBJECTS BY IMAGE MEASUREMENT APPROACH (IMA)

To further determine the location and size of the marine debris, it is necessary to locate the target object's position and

dimensions after identifying it. This study employs the Image Measurement Approach (IMA) proposed by Tsai et al. for image depth positioning technology [39]. IMA divides object positioning and dimension measurement into three steps:

The robot's camera transforms the horizontally captured image into a top-down view using simple Inverse Perspective Mapping (IPM), as illustrated in Figure 3. We assume that the target object's pixel coordinates in the top-down view are $T(120, 130)$, commencing from the top-left corner at $(0, 0)$. By establishing a correspondence between pixel intervals in the image and distances in the real world, we can further calculate the absolute position of the target object in the real world. For instance, if the pixel interval corresponds to a real-world distance of 0.1 cm, then the absolute position of the target object would be (12 m, 13 m).

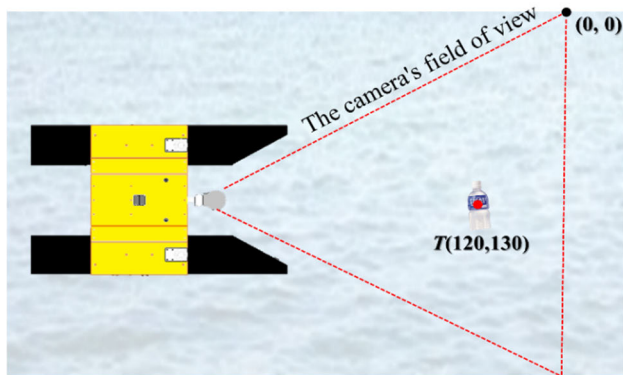


FIGURE 3. A conceptual diagram of the image after IPM transformation.

After determining the target object's position, calculating its size is critical to ensuring the robot can successfully traverse it for collection. The height of the object is calculated using a triangulation method to measure the object's height (OH), where the camera's elevation angle θ must satisfy Equation (8), with FOV_v representing the camera's vertical field of view. We select the highest point TP and lowest point BP of the object in the image, as shown in Figure 4, and assume the pixel coordinates of TP are (TP_x, TP_y) and BP are (BP_x, BP_y) , with the top-left corner of the image at $(0, 0)$. We then compute the camera's optical axis passing through TP and BP, projected onto the plane lengths $Y1$ and $Y2$, using Equations (9) and (10), based on the shooting angle θ .

Finally, Equation (11) shows how to calculate the object's height based on the lengths $Y1$ and $Y2$ projected onto the plane from TP and BP, as well as the height of the robot's camera.

$$\theta < 90 - \frac{FOV_v}{2} \quad (8)$$

$$\delta = \theta + \left(\frac{H}{2} - j \right) \times \left(\frac{FOV_v}{H} \right) \quad (9)$$

$$Y = h \times \tan(\delta) \quad (10)$$

$$OH = h \times \left(\frac{Y2 - Y1}{Y2} \right) \quad (11)$$

Similar to calculating the object's height, we can also determine its width (OW) using a similar method. Here, q represents the object's vertical coordinate in the image, and FOV_H denotes the camera's horizontal field of view. Equations (12) and (13) allow us to obtain the lengths $X1$ and $X2$, projected through the optical axes of the two points, by selecting the pixel coordinates of the object. Finally, by calculating the distance between the two points, we can determine the width of the object.

$$\beta = \left(q - \frac{W}{2} \right) \times \left(\frac{FOV_H}{W} \right) \quad (12)$$

$$X = Y \times \tan(\beta) \quad (13)$$

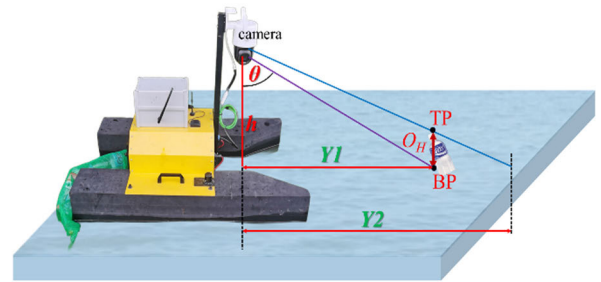


FIGURE 4. The triangulation method for calculating the target object's height and width.

C. PATH PLANNING OF DyNav

To adapt to the ocean's dynamic environment, we must consider the constant shifting of floating debris as the robot moves. Therefore, we need to consider the movement status of the floating debris to determine the next target location to traverse. After performing a Breadth-First Search (BFS) traversal, we obtain a set of candidate sequences, with each target element undergoing calculations by the path planning program. We select the target with the lowest total cost as the next target location for the robot to traverse. The purpose of using BFS is to prioritize the removal of marine debris located further away from the robot, as it would otherwise require the robot to traverse excessively long paths. Once we obtain a set of candidate sequences, we prioritize processing the marine debris within the sequence. Since marine debris is dynamic, after each traversal (collection) of a piece of marine debris, another BFS is conducted to identify a new candidate sequence until no new targets appear or until the designated work time is reached, and so on.

After generating the candidate sequence, we carry out the path planning process for the marine debris within the sequence. We have proposed a heuristic path-planning algorithm, named DyNav. In the DyNav algorithm, we have defined the following heuristic function, as shown in equation (14).

$$C_{total}(G_i) = C_{real}(G_i) + C_{esti}(G_i) + C_{adapt}(G_i) \quad (14)$$

where $C_{total}(G_i)$ is the **total cost** from the current position G_i , when selecting the next marine debris to traverse, the robot will choose the one with the lowest total cost. $C_{real}(G_i)$ represents the **actual cost** from the starting point G_s to the current position G_i ; $C_{esti}(G_i)$ denotes the **estimated cost** from the current position G_i to the target position G_k ; and $C_{adapt}(G_i)$ is the **adaptive cost** from the current position G_i to the target position G_k .

Equation (15) calculates the actual cost, which is the cost of the robot's journey from the starting point G_s to the current position of the marine debris G_i .

$$C_{real}(G_i) = \sum_{j=1}^i dist(G_s, G_j) \quad (15)$$

Equation (16) outlines the calculation method for the estimated cost, where G_k represents the target position and G_i represents the current position.

$$C_{esti}(G_i) = \sum_{j=i+1}^k dist(G_i, G_j) \quad (16)$$

where $dist(G_i, G_j) \leq td$ The adaptive cost, as shown in equation (17), is the final component in computing the overall priority when considering time and power costs. The direction and speed of the water current determine the travel time of the robot from the current position G_i to the target position G_k , represented by $t_{i,k}$. If the water current is in the downstream direction, the robot is in a more favorable state in terms of salvage time and energy consumption; conversely, in an upstream current, it is in the worst state.

$$C_{adapt}(G_i) = t_{i,k} + E_{i,k} \quad (17)$$

Equations (18) and (19) respectively represent the calculation methods for the travel time in downstream and upstream directions as the robot moves towards the target grid. Here, s (distance per unit time) signifies the robot's speed of movement, μ (distance per unit time) denotes the water current speed, d stands for the distance the robot travels from grid a to grid b , and θ (in degrees) represents the heading angle. A flowmeter and the Beidou satellite system can provide flow velocity and heading angle information.

Equation (18) can calculate the robot's travel time $t_{i,k}$ if the water current aligns with the robot's heading angle, which indicates downstream flow:

$$t_{i,k} = \frac{d}{T_e} \quad (18)$$

where $T_e = s + \mu \times \cos(\theta)$

Equation (19) can calculate the robot's travel time $t_{i,k}$ if the water current is opposite to the robot's heading angle, indicating an upstream flow:

$$t_{i,k} = \frac{d}{T_e} \quad (19)$$

where $T_e = s - \mu \times \cos(\theta)$

Assuming the robot moves from the current position G_i to G_k , the energy consumption $E_{i,k}$ can be considered as the change in the robot's kinetic energy during movement. The robot's velocity variation can represent the change in kinetic energy, with an increase or decrease in kinetic energy corresponding to the robot's energy consumption.

We can calculate the robot's energy consumption, $E_{i,k}$, using the following equation:

$$E_{i,k} = \frac{1}{2} \omega (v_f^2 - v_i^2) \quad (20)$$

where ω is the mass of the robot. The v_i is the robot's initial velocity, and the v_f is its final velocity. We calculate the values of v_i and v_f based on the robot's movement in the water current, along with factors like water current speed and heading angle.

We can adjust the velocities v_i and v_f based on the direction of the water current and the robot's heading angle, considering the influence of the water current. We can calculate the robot's final velocity v_f as shown in equation (21) if the water current aligns with the robot's heading angle, indicating downstream flow.

$$v_f = V_{robot} + V_{water} \quad (21)$$

Equation (22) expresses the robot's final velocity v_f when the water current is opposite to the robot's heading angle, indicating an upstream flow.

$$v_f = V_{robot} - V_{water} \quad (22)$$

To illustrate the method of calculating the adaptive cost, let's consider the following scenario:

Assuming the robot's movement speed is 4 km/h, the water current speed is 2 km/h, the distance from grid a to grid b is 500 m, and the heading angle is 30 degrees.

Let's first calculate the travel time $t_{i,k}$ and the energy consumption $E_{i,k}$ in the downstream flow scenario:

- 1) Calculate the travel time $t_{i,k}$ in the downstream flow scenario.

$$\begin{aligned} T_e &= s + \mu \times \cos(\theta) = 4 + 2 \times \cos(30^\circ) \\ &= 4 + 2 \times \frac{\sqrt{3}}{2} = 4 + \sqrt{3} \\ &\approx 5.73 \text{ km/h} \end{aligned}$$

$$t_{i,k} = \frac{500 \text{ m}}{5.73 \text{ km/h}} \approx 0.087 \text{ hours}$$

- 2) Calculate the energy consumption $E_{i,k}$ based on the energy consumption formula (20):

$$\begin{aligned} v_i &= 4 \text{ km/h} \\ v_f &= 4 + 2 = 6 \text{ km/h} \\ E_{i,k} &= \frac{1}{2} \omega (v_f^2 - v_i^2) = \frac{1}{2} \omega (6^2 - 4^2) \\ &= \frac{1}{2} \omega (36 - 16) = 10\omega \end{aligned}$$

Next, calculate the travel time $t_{i,k}$ and the energy consumption $E_{i,k}$ of the robot in the upstream flow scenario.

- 1) Calculate the travel time $t_{i,k}$ in the upstream flow scenario.

$$\begin{aligned} T_e &= s + \mu \times \cos(\theta) = 4 - 2 \times \cos(30^\circ) \\ &= 4 - 2 \times \frac{\sqrt{3}}{2} = 4 - \sqrt{3} \approx 2.27 \text{ km/h} \\ t_{i,k} &= \frac{500\omega}{2.27 \text{ km/h}} \approx 0.220 \text{ hours} \end{aligned}$$

- 2) Calculate the energy consumption $E_{i,k}$ under reverse flow conditions according to the energy formula (20), which is also equal to 10ω .

Therefore, under the downstream conditions, the robot's adaptation cost $C_{adapt}(G_i) = t_{i,k} + E_{i,k} = 0.087 + 10\omega$. Under upstream conditions, the adaptation cost is $C_{adapt}(G_i) = t_{i,k} + E_{i,k} = 0.22 + 10\omega$. Because upstream conditions require longer travel time, the total adaptation cost increases accordingly.

D. THE ENTIRE OPERATION PROCESS OF DyNav

The robot departs from the starting point G_s , and here are the complete operational steps of DyNav:

Step 1: If the robot finds no target within the detection range, it moves in a random direction. Otherwise, BFS generates candidate target positions, assuming a sequence length of k . The elements in the sequence are sequentially selected as the target positions G_i for the robot to traverse.

Step 2: As shown in Equation (14), calculate the total cost for the target position at G_i .

Step 3: Return to step two if any elements in the sequence remain uncalculated; otherwise, move on to step four.

Step 4: The robot selects the target object with the lowest total cost as the destination for traversal. The robot moves to the target location with the lowest cost and traverses floating marine debris. YOLOv7 is used to identify the target objects, and IMA determines their positions and sizes.

Step 5: Once you reach the designated time, continue with the scheduled work; if not, terminate the program and proceed to the first step. If exceptional circumstances arise, recalculate the total cost of the target object or submit a repair request to the manager.

V. SYSTEM TESTING AND SIMULATION EXPERIMENTS

A. SIMULATION EXPERIMENTS AND PARAMETER SETTINGS

This study used a Windows 11 operating system with 16GB of DDR4-3000 memory in the simulation experiments. We employed Java, MATLAB, and Gnuplot as auxiliary tools for plotting and complex computations.

To prove that DyNav works, simulation tests were done on five different path planning algorithms: DyNav (the one proposed in this study), Greedy, RRT, SMURF [17], and U* [26]. RRT and Greedy have the advantage of being simple. In the Greedy method, when the robot detects marine debris,

TABLE 3. The pseudo code of DyNav.

Input: Starting point G_s , sequence of candidate target positions, sequence length k	
Output: The robot traverses the target position G_i and updates the information on marine debris	
1:	While the target object is detected && has not reached the desire time do
2:	If no target object was found then
3:	robot moves in a random direction;
4:	Else
5:	Generate a sequence of candidate target positions with a length of k ;
6:	While There are elements in the sequence that have not been calculated do
7:	G_i = take out the elements of the sequence;
8:	calculateTotalCost(G_i);
9:	End while
10:	End If
11:	Select the target position with the lowest total cost for the robot to traverse;
12:	The robot moves towards the target position with the lowest cost and retrieves marine debris;
13:	Detect the target object using YOLOv7 and locate the target object with IMA;
14:	If anomaly detected then
15:	recalculate the total cost of the target object;
16:	Submit a repair request to the manager;
17:	End If
18:	End while

it directly navigates to the debris location for collection. In actual marine debris collection, there are numerous methods that involve mechanically grabbing or salvaging the target objects. In the simulation experiments, the term 'collection' refers to the process of the robot traversing marine debris. We assume in the simulation experiments that the robot successfully collects ocean garbage when the distance between it and the marine debris is less than 0.1 meters. In the actual testing, because the collection chamber is positioned in the middle of the robot, the robot must traverse and pass through the location of the marine debris for it to enter the robot's collection chamber.

During the detection process, the robot records the ID and location information of the marine debris in a queue, removing them sequentially according to a 'first-in, first-out' strategy. Another method employs the greedy approach to reduce the robot's time and power costs. The robot uses the location information of marine debris in the queue and first selects the nearest debris for traversing.

We conducted four sets of experiments with the following content for each: (A) varying the robot's movement speed; (B) total number of deployed marine debris; (C) obstacle sizes; and (D) altering ocean current speeds. Table 4 lists the experimental conditions and parameters for the simulation experiments.

Equation (23) defines the method for calculating the average path length, Avg_{leng} , where ex represents the total number of runs. If there is an edge between grid i and grid j , $\eta_{ij} = 1$; otherwise, $\eta_{ij} = 0$. The formula for calculating the average path length is as follows: for each run, calculate the distance

TABLE 4. Parameter settings for simulation experiments.

Item	value
Area size	500 m *500 m
Number of robots	1
Robot's movement speed	5 m/s -14 m/s
Total number of marine debris	10-100
Effective detection range	5 m
Obstacle of size	10%-55%
Ocean velocity	10 m/s-20 m/s
Simulation time	3000 second

between grids G_i and G_j multiplied by η_{ij} (If there is an edge, the value is 1, and if not, it is 0.), then sum these values and divide by the total number of runs, ex .

$$Avg_{leng} = \left(\sum_{m=1}^{ex} \sum_{i=1}^{n+1} \sum_{j=1}^{n+1} dist(G_i, G_j) \eta_{ij} \right) \times \frac{1}{ex}, \quad (23)$$

where $\eta_{ij} = \{0, 1\}$.

Equation (24) defines the average amount of marine debris collected, denoted as GN_{avg} , where GN_m represents the number of marine debris collected in the m -th experiment. Additionally, when the distance between the robot and the marine debris is less than 1 meter, it is considered a successful collection; otherwise, it is considered a failed collection, where ex denotes the total number of runs.

$$GN_{avg} = \sum_{m=1}^{ex} GN_m \times \frac{1}{ex} \quad (24)$$

To make the simulation experiments closer to reality, in addition to simulating the marine environment using Orcaflex software, this study also utilized an ocean current kinematic model [38] for the motion patterns of floating marine debris. Equation (25) represents the kinematic model of ocean currents.

$$\begin{aligned} V_x &= k_1 \lambda V \sin(k_2 x) \cos(k_3 y) \\ &\quad + k_1 \lambda \cos(2k_1 t) + k_4 \\ V_y &= -\lambda V \cos(k_2 x) \sin(k_3 y) + k_5 \end{aligned} \quad (25)$$

In this case, V_x represents the velocity on the x-axis, V_y represents the velocity on the y-axis, and k_1, k_2, k_3, λ , and V are variables closely related to environmental factors such as tides and water depth. These parameters vary in different environments, with specific changes detailed in Table 5. Variations in these parameters have a direct impact on the motion and behavior of underwater robots, making it critical to adjust these parameters in different environments.

B. VARYING ROBOT'S MOVEMENT SPEED

In this set of experiments, we verify the impact of the robot's movement speed on the robot's total path length and garbage collection efficiency. The robot's movement speed ranges from 5 m/s to 14 m/s. We fix the ocean current speed at

TABLE 5. Ocean current model parameter settings.

Parameters	Mean	std
k_1, k_2	π	0.1π
k_3	2π	0.2π
k_4, k_5	1	0.1
V	1	0.1
λ	0.5~3.0	0.05~0.3

3 m/s, with obstacles covering 30% of the total area. The total number of marine debris items remains constant at 50.

Table 6 displays the experimental findings for the total amount of marine debris gathered using five algorithms. The results consistently show that the DyNav algorithm achieves the highest garbage collection quantity in all cases, demonstrating outstanding garbage collection capability. In comparison, the greedy algorithm tends to collect a lower amount of debris in most situations, possibly due to its greedy strategy limitation, which selects only the most favorable scenarios for decision-making without comprehensive consideration, thereby affecting efficiency. In certain scenarios, the RRT algorithm performs well, but it also exhibits lower collection quantities. Its random path generation approach, while effective in specific environments or obstacle handling, often exhibits limitations. In most cases, the SMURF algorithm performs reasonably well, but it still falls short of the DyNav algorithm's collection efficiency. This is because the SMURF algorithm follows a pre-planned and comprehensive path planning strategy, which, within a limited simulation time, restricts the robot from covering the entire area, leaving room for improvement in efficiency and collection capability. On the other hand, the U* algorithm achieves the maximum garbage collection quantity in all scenarios, highlighting its excellent path planning and garbage collection capabilities.

TABLE 6. The impact of robot movement speed on the quantity of garbage collection.

MS\AL	DyNav	Greedy	RRT	SMURF	U*
5	50	36.16	14.59	29.72	50
6	50	39.77	13.13	30.06	50
7	50	42.27	16.6	31	50
8	50	40.78	15.03	30.79	50
9	50	43.4	17.34	31.26	50
10	50	42.79	18.67	34.24	50
11	50	42.72	13.54	32.54	50
12	50	44.25	15.34	32.17	50
13	50	39.69	14.67	33.16	50
14	50	43.9	20.61	32.42	50

MS: robot's movement speed. AL: algorithms

Figure 5 illustrates how robot movement speed affects the total path length. According to the experimental results, as the robot's movement speed increases, its total path length also increases within a limited time frame. DyNav and U*

algorithms exhibit similar trends, as both consider the direction and speed of ocean currents for efficient path planning. In contrast, the Greedy algorithm selects the nearest marine debris, resulting in longer total path lengths compared to DyNav and U*. The RRT algorithm employs a random exploration approach. On the other hand, SMURF follows a static full-coverage path planning strategy, which is less efficient in dynamic ocean environments, leading to the longest path lengths.

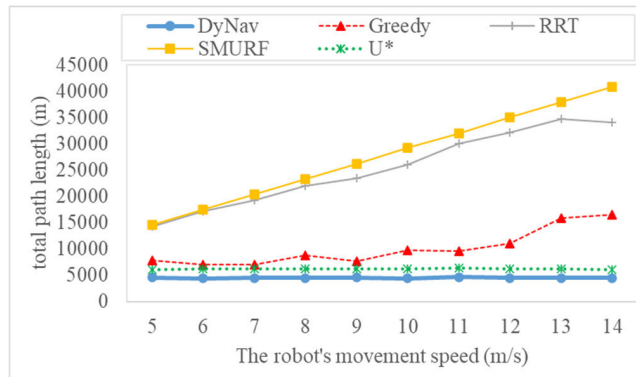


FIGURE 5. The experimental result demonstrates the impact of the robot's movement speed on the total path length.

C. CHANGING THE SIZE OF OBSTACLES

In this set of experiments, we investigated the impact of obstacle size on robots' performance in marine debris collection and total path length. We fixed the robot's movement speed at 10 m/s and the ocean current speed at 3 m/s. We varied the proportion of obstacle area to total area from 10% to 55%. We kept the total number of marine debris items constant at 50.

Table 7 shows the experimental results of the total amount of marine debris collected by five algorithms. The results indicate that as the proportion of obstacles increases, greedy and RRT exhibit a gradual decrease in the amount of garbage collected. This is due to the increased obstacles causing restrictions on robot movement paths, resulting in a reduction in garbage collection efficiency. In contrast, the DyNav algorithm maintains a relatively stable garbage collection quantity in most cases, suggesting better adaptability to obstacles. In most cases, the SMURF algorithm outperforms DyNav in terms of collection efficiency. This is because SMURF follows a pre-planned and comprehensive path planning strategy, which limits the robot's ability to cover the entire area within the simulation time, leaving room for improvement in efficiency and collection capability. The U* algorithm maintains a stable garbage collection quantity across different obstacle proportions, indicating potential good resistance to interference or effective path planning. Worth further investigation.

In Figure 6, as the proportion (size) of obstacles increases, the robot faces greater challenges in finding viable paths. When considering the direction and speed of ocean currents,

DyNav and U* demonstrate relatively stable performance, whereas Greedy, when focusing excessively on the nearest target, leads to an increase in path length. Notably, we observe a slight decrease in path length when the obstacle proportion reaches 30%. The possible reason is that as the obstacle area increases, the number of available paths decreases relatively, thereby increasing the probability of selecting better paths and consequently reducing the total path length. The RRT method adopts random exploration, which, despite some instability, may also discover better paths in certain situations. On the other hand, while SMURF's full-coverage static path planning performs well in stable environments, its efficiency is not as high as other methods in dynamic marine environments. This suggests that considering more factors, such as ocean currents and obstacle distribution, is crucial in dynamic environments.

TABLE 7. The impact of obstacle size on the amount of garbage collected.

ROA\AL	DyNav	Greedy	RRT	SMURF	U*
10 %	50	41.62	13.67	38.79	50
15 %	50	39.52	16.13	36.48	50
20 %	50	39.27	16.26	35.33	50
25 %	50	40.54	17.41	33.8	50
30 %	50	45.08	15.89	33.32	50
35 %	50	44.02	17.14	31.91	50
40 %	50	43.56	18.21	31.79	50
45 %	50	42.27	20.4	30.68	50
50 %	50	41.73	17.66	30.22	50
55 %	50	42.35	19.28	29.62	50

ROA: ratio of obstacle to entire area. AL: algorithms

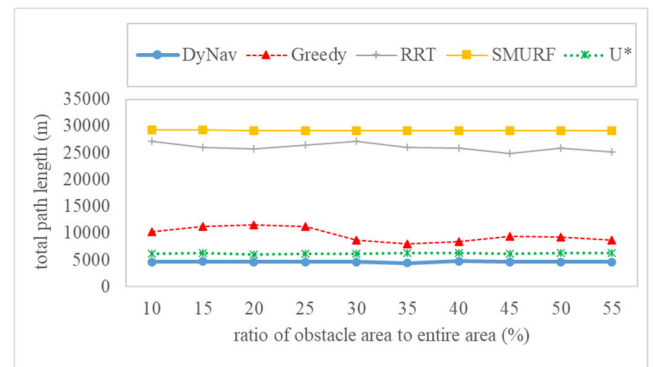


FIGURE 6. The experimental result demonstrates the impact of obstacle size on the total path length.

D. CHANGES IN THE TOTAL QUANTITY OF MARINE DEBRIS

In this set of experiments, we introduced varying quantities of marine debris to analyze the performance of the robot in marine debris collection and its impact on the total path length. The robot's movement speed was fixed at 10 m/s. The ocean velocity was set at 3 m/s, with obstacles covering a

fixed proportion of 30% of the total area. The total amount of marine debris introduced ranged from 10 to 100.

Table 8 displays the experimental results of the total amount of marine debris collected by five algorithms. The results indicate that as the quantity of marine debris introduced increases, most algorithms show a corresponding rise in the total amount of marine debris collected. However, there are significant variations in the performance of each algorithm at different levels of marine debris quantities. For instance, with the increase in the quantity of marine debris introduced, the RRT algorithm initially shows a high collection total, which then gradually declines. On the other hand, the U* algorithm consistently performs well at lower quantities, increasing the collection total as the number of marine debris increases. The data also reveals notable changes; for instance, the Greedy algorithm sharply reduces the collection total at an input of 80 marine debris items, while the RRT algorithm also significantly reduces the collection total.

Figure 7 shows how the total quantity of marine debris introduced affects the total path length. According to the experimental results, as the quantity of marine debris increases, the robot needs to traverse longer distances to complete the task of collecting marine debris. Initially, the DyNav and U* algorithms, which both consider the direction and speed of ocean currents for efficient path planning, exhibit a similar trend. However, as the quantity of marine debris increases, we observe a gradual divergence between the two algorithms until the total number of marine debris reaches 60, at which point the difference stabilizes. In contrast, the Greedy algorithm selects the nearest marine debris, resulting in longer total path lengths compared to DyNav and U*. The difference between the Greedy algorithm and U* gradually widens as the total number of marine debris reaches 60. The RRT algorithm employs a random exploration approach, leading to the fastest growth in path length to accomplish the task. On the other hand, SMURF follows a static full-coverage path planning strategy, which is less efficient in dynamic marine environments, resulting in the longest path lengths.

TABLE 8. The experimental results show the quantity of garbage collected by the five algorithms.

MMD\ AI	DyNav	Greedy	RRT	SMURF	U*
10	10	9.92	9.04	6.98	10
20	20	18.62	15.86	13.49	20
30	30	27.73	17.51	20.18	30
40	40	33.43	17.02	27.07	40
50	50	44.34	14.17	34.19	50
60	60	49.87	13.93	38.9	60
70	70	55.95	13.43	44.07	70
80	80	56.85	8.64	50.63	80
90	90	59.77	10.91	60.12	90
100	100	63.16	27.35	69.11	100

MMD: total number of marine debris deposited. AI: algorithms

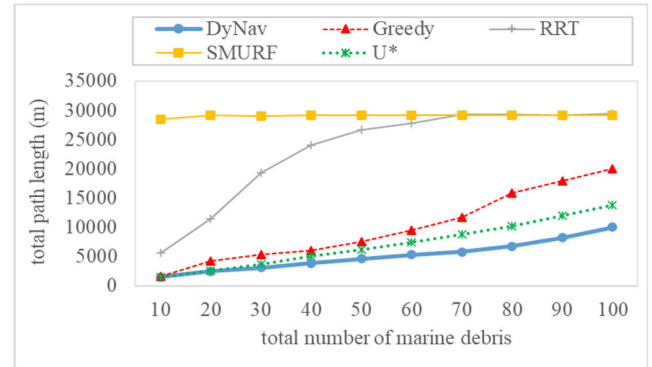


FIGURE 7. The total number of deployed marine debris affects the total path length of the robot.

E. VARIATIONS IN OCEAN VELOCITY

In this set of experiments, we validated the impact of ocean velocity on the robot's performance and total path length in marine debris collection. The robot's fixed movement speed is 10 m/s. The ocean velocity ranged from 2.2 m/s to 4.0 m/s, with the obstacle area occupying a fixed proportion of 30% of the total area. The total number of marine debris items remained fixed at 50.

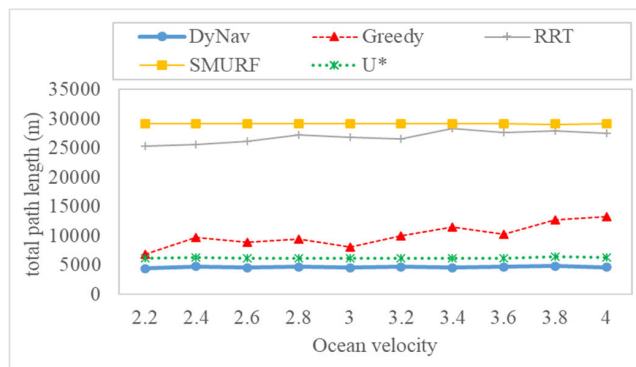
Table 9 presents the experimental results for the total number of marine debris collected by five algorithms. The results indicate that with increasing ocean velocity, Greedy and RRT show a gradual decrease in the number of debris collected. This is due to the faster drift of marine debris, which limits the robot's effectiveness in collecting them. In contrast, DyNav and U* algorithms maintain relatively stable debris collection numbers in most cases, suggesting better adaptability to obstacles. The SMURF algorithm demonstrates relatively stable performance in most cases but still lags behind DyNav in collection efficiency. Because SMURF employs a pre-planned and exhaustive path planning strategy, the robot cannot cover the entire area within the limited simulation time, leaving room for improvement in efficiency and collection capability. Notably, the U* algorithm keeps the same number of pieces of debris collected even when there are different numbers of obstacles. This suggests that it is very good at blocking interference or planning routes, which calls for more research.

Figure 8 shows how the ocean velocity items affect the total path length. According to the experimental results, as the ocean velocity increases, the robot needs to travel longer distances to complete the task of collecting marine debris. The drifting distance of marine debris increases. DyNav and U* exhibit similar behaviors, both considering the direction and ocean velocity for efficient path planning. On the other hand, Greedy selects the nearest marine debris, resulting in longer total path lengths compared to DyNav and U*. As the total number of marine debris items rises, the total path length gradually increases. RRT uses a random exploration approach, maintaining a total path length of around 25,000 meters to accomplish the task. SMURF, which is a static

TABLE 9. The experimental results demonstrate the impact of ocean velocity on the amount of garbage collected.

OV\AI	DyNav	Greedy	RRT	SUMRF	U*
2.2	50	46.33	21.92	32.42	50
2.4	50	43.4	20.22	32.55	50
2.6	50	43.07	17.64	32.82	50
2.8	50	43.39	15.69	32.38	50
3.0	50	43.44	17.86	32.77	50
3.2	50	41.48	15.68	33.22	50
3.4	50	38.88	14.1	32.16	50
3.6	50	41.04	11.5	32.15	50
3.8	50	38.67	11.35	32.88	50
4.0	50	36.74	14.25	31.8	50

OV: ocean velocity. AI: algorithms

**FIGURE 8.** The effect of ocean velocity on the robot's total path length.

exhaustive path planning method, is inefficient in dynamic marine environments, resulting in the longest path lengths.

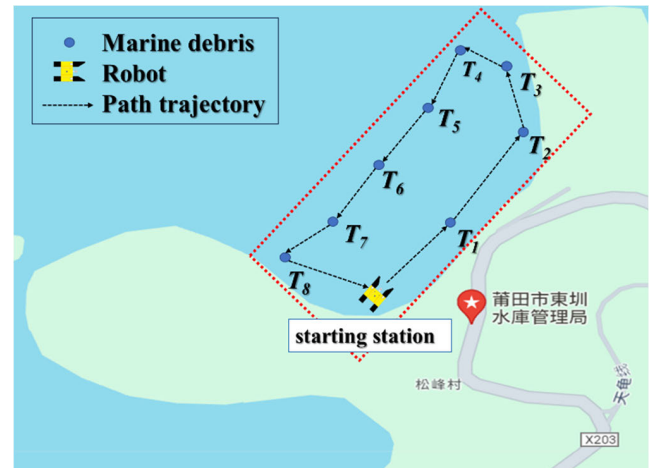
VI. ROBOT'S ACTUAL VALIDATION AND TESTING

In the real-world validation, we conducted field tests at the Dongzhen Reservoir in Putian City, Fujian Province. The area enclosed by the red border, approximately $100m \times 30m$ in size, as shown in Figure 9, served as our testing region. We randomly placed 20 plastic bottles in the reservoir within 1–5 meters from the red border and evaluated the robot's capture ability and the total path length traveled by the robot. Table 10 lists the experimental parameters. Each experiment lasted for 30 minutes, with some repeated twice to obtain an average value as the final validation result.

We conducted tests on three path planning algorithms: DyNav, which was proposed in this study, Greedy, and U*. The experiments are as follows: (A) changing the robot's movement speed to observe the robot's garbage collection efficiency; and (B) testing the robot's positioning and carrying capacity.

A. ROBOT'S GARBAGE COLLECTION EFFICIENCY

The experimental results of the three algorithms in Table 11 demonstrate that both DyNav and U* algorithms excel in collecting marine debris, achieving an efficiency of 100%.

**FIGURE 9.** Dongzhen Reservoir and Environmental Deployment.**TABLE 10.** Experimental conditions and setups.

Item	value
Area size	100 m × 30 m
number of robots	DaYu No. 1 × 1
effective detection range	5 m
number of PET bottles deposited	20
robot movement speed	5-10 km/h

In contrast, the efficiency of the Greedy algorithm is only around 60%. Within the limited time frame, the Greedy algorithm failed to complete the task within 30 minutes, indicating its lack of efficiency in planning. The success of DyNav and U* algorithms lie in their optimized planning strategies, allowing effective robot navigation to minimize costs or time for task completion. The robot can swiftly and accurately collect marine debris thanks to DyNav's efficient path planning, which combines ocean current information with the proposed heuristic function in this study. On the other hand, the U* algorithm utilizes genetic algorithms and ocean current information to assist the robot in quickly finding the best path. In comparison, the Greedy algorithm may adopt simple and local optimal strategies in complex environments, leading to suboptimal global path planning and lower collection efficiency. In this experiment, the Greedy algorithm may have overlooked the global optimal solution, resulting in inefficient resource and time utilization.

B. ROBOT'S POSITIONING AND CARRYING CAPACITY

This experiment tested the positioning and load-carrying capacity of DaYu No. 1. A BeiDou satellite receiver equips DaYu No. 1. Table 12 shows the test results. The results reveal a maximum latitude value of 2238.81628, a minimum value of 2238.81438, a maximum deviation of 2.1298527608451 meters, and a minimum deviation of 1.29014723952423 meters between these two values. This indicates that in latitude measurement, the maximum deviation is slightly higher than the minimum deviation, suggesting

TABLE 11. The experimental results demonstrate the effectiveness of the three algorithms in real-world validation.

	DyNav	Greedy	U*
5	20	14.5	20
6	20	15	20
7	20	16.5	20
8	20	16	20
9	20	17.5	20
10	20	17.5	20

potential measurement errors or system inaccuracies. For longitude, we measured a maximum value of 11404.38621 and a minimum value of 11404.38514, with a maximum deviation of 1.15017791540595 meters and a minimum deviation of 0.775822085051914 meters. Similar to latitude, the maximum deviation in longitude is slightly higher than the minimum deviation, indicating some level of variation or uncertainty in longitude measurement.

Various factors, such as the accuracy of measuring devices, changes in environmental conditions, and signal interference, can influence positioning errors. In future research, we can explore different positioning technologies or improve measurement methods to reduce positioning errors, ensuring the accuracy and reliability of data for various applications.

TABLE 12. The Dayu No. 1 positioning accuracy test results are now available.

latitude	maximum 2238.81628	minimum 2238.81438
	Maximum deviation (meter) 2.1298527608451	Minimum deviation (meter) 1.29014723952423
longitude	maximum 11404.38621	minimum 11404.38514
	Maximum deviation (meter) 1.15017791540595	Minimum deviation (meter) 0.775822085051914

In the load-carrying capacity test, we selected a 6-kilogram weight as the robot's load. We observed that the robot could smoothly perform turning, forward, and reverse operations while carrying this weight. This indicates that the robot can maintain stable mobility even when subjected to a certain load. As robots may need to carry various loads in practical applications, ensuring that they can effectively carry and operate these loads is crucial. The results of the load-carrying capacity test demonstrate that the robot's mechanical structure and power system can handle a load of 6 kilograms. Furthermore, the robot's motion control system can effectively manage this additional burden, maintaining its normal operational capabilities. This reinforces the robot's stability and reliability when carrying loads in practical applications.

C. LESSONS LEARNED AND LIMITATIONS DISCUSSION

By using our suggested DyNav algorithm in the field of robots that collect marine debris and carefully comparing it to other algorithms like Greedy, RRT, SMURF, and U*, we have

seen that DyNav has clear advantages in some areas. Firstly, utilizing the DyNav algorithm, we have noticed that, compared to other algorithms, the robot's path planning during debris collection can be more efficient and quicker, leading to overall improved efficiency. Secondly, DyNav demonstrates adaptability in complex environments, flexibly addressing various marine conditions.

However, we must acknowledge some limitations of the DyNav algorithm. For instance, in the face of extreme weather or oceanic situations, further enhancements and optimizations are required to improve the algorithm's stability and reliability. In the future, we plan to conduct more practical operations and refinements to enhance its capabilities.

VII. CONCLUSION

In this study, we focus on marine debris by creating and validating DaYu No. 1, a cleaning robot, and the DyNav algorithm. Through simulations and tests, DaYu No. 1 proves effective for marine debris collection, which is critical for marine environment protection. Compared to other methods like Greedy, RRT, SMURF, and U*, DyNav excels in debris cleanup. Future works include enhancing DaYu No. 1's performance. We aim to refine heuristic functions by taking into account factors such as obstacles and wind, ensuring better path planning adaptability. To cope with dynamic marine conditions, we'll adjust weights based on real-time changes. We also intend to implement multi-objective optimization, looking beyond just cost to factors like energy use for comprehensive planning. By analyzing data, we'll improve heuristics for smarter path planning. Our work not only delivers results but also guides future research. We believe DaYu No. 1 and DyNav will significantly aid marine environment protection and cleaning efforts, fostering cleaner seas.

REFERENCES

- [1] J. Guo, X. Huo, S. Guo, and J. Xu, "A path planning method for the spherical amphibious robot based on improved A-star algorithm," in *Proc. IEEE Int. Conf. Mechatronics Autom. (ICMA)*, Takamatsu, Japan, Aug. 2021, pp. 1274–1279.
- [2] Y. Xiu-xia, C. Wei-yi, Z. Yi, F. Guo-wei, and Y. Xuan, "Mobile robot path planning in complex environment," in *Proc. IEEE Int. Conf. Unmanned Syst. (ICUS)*, Beijing, China, pp. 426–431, Oct. 2019.
- [3] J. Zhang, Z. Wang, G. Han, Y. Qian, and Z. Li, "A collaborative path planning method for heterogeneous autonomous marine vehicles," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 1465–1480, Jan. 2024.
- [4] S. Arora, C. Lund, R. Motwani, M. Sudan, and M. Szegedy, "Proof verification and the hardness of approximation problems," *J. ACM*, vol. 45, no. 3, pp. 501–555, May 1998.
- [5] Y. Wang, Y. Zhao, Y. Wu, S. Zhang, and J. Wang, "A multi-sensor intelligent surface garbage cleaning robot," in *Proc. IEEE Int. Conf. Mechatronics Autom. (ICMA)*, Takamatsu, Japan, Aug. 2021, pp. 797–801.
- [6] A. Rincon-Suarez, M. Rubin-Falfan, E. Sanchez-Sanchez, G. Trinidad-García, H. Juárez, E. Rosendo, T. Diaz, and G. G. Salgado, "Design and construction of an autonomous cleaner robot, for an aquatic environment," in *Proc. Electron., Robot. Automat. Mech. Conf. (CERMA)*, 2018, pp. 482–487.
- [7] Md. S. Ahsan, H. Mobarak, and K. N. Chy, "Design and construction of a radio controlled floating waste collector robot," in *Proc. 11th Int. Conf. Electr. Comput. Eng. (ICECE)*, Dec. 2020, pp. 105–108.

- [8] M. Turesinin, A. M. H. Kabir, T. Mollah, S. Sarwar, and Md. S. Hosain, "Aquatic iguana: A floating waste collecting robot with IoT based water monitoring system," in *Proc. 7th Int. Conf. Electr. Eng., Comput. Sci. Informat. (EECSI)*, Oct. 2020, pp. 21–25.
- [9] J. Panyavaraporn, N. Chaimongkol, N. Limsomnuek, W. Wasayangkul, N. Charoenwattana, and P. Horkaew, "Dual mode controlled water surface garbage collecting robot by using embedded deep learning," in *Proc. 18th Int. Conf. Electr. Engineering/Electronics, Comput., Telecommun. Inf. Technol. (ECTI-CON)*, May 2021, pp. 14–17.
- [10] Y. Bai, S. Yang, Y. Wu, Y. Chen, S. Pan, and Y. Deng, "Expansible surface waste cleaning robot," in *Proc. 5th Int. Conf. Comput. Commun. Syst. (ICCCS)*, May 2020, pp. 378–383.
- [11] S. Kong, M. Tian, C. Qiu, Z. Wu, and J. Yu, "IWSCR: An intelligent water surface cleaner robot for collecting floating garbage," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 10, pp. 6358–6368, Oct. 2021.
- [12] Azure Fighte. Accessed: May 3, 2022. [Online]. Available: <https://azurealliance.org/azure-fighter/>
- [13] WasteShark. Accessed: May 1, 2022. [Online]. Available: <https://www.wevolver.com/wevolver.staff/wasteshark>
- [14] Mr. Trash Wheel. Accessed: May 3, 2022. [Online]. Available: <https://www.mrtrashwheel.com/>
- [15] Seabin Project. Accessed: May 3, 2022. [Online]. Available: <https://seabinproject.com/>
- [16] TOC. Accessed: May 3, 2022. [Online]. Available: <https://theoceancleanup.com/>
- [17] J. Zhu, Y. Yang, and Y. Cheng, "SMURF: A fully autonomous water surface cleaning robot with a novel coverage path planning method," *J. Mar. Sci. Eng.*, vol. 10, no. 11, p. 1620, Nov. 2022.
- [18] P.-F. Xu, Y.-X. Ding, and J.-C. Luo, "Complete coverage path planning of an unmanned surface vehicle based on a complete coverage neural network algorithm," *J. Mar. Sci. Eng.*, vol. 9, no. 11, p. 1163, Oct. 2021.
- [19] Z. Wang, M. J. Er, and H. Gong, "An improved a path planning algorithm of autonomous underwater vehicles," in *Proc. 5th Int. Conf. Intell. Auto. Syst. (ICoIAS)*, Sep. 2022, pp. 186–190.
- [20] Z. Lin, K. Wu, R. Shen, X. Yu, and S. Huang, "An efficient and accurate A-Star algorithm for autonomous vehicle path planning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 9003–9008, Jun. 2024.
- [21] M. Su, Y. Fei, K. He, and F. Liu, "Global path planning of AUV based on GA-SQP algorithm," in *Proc. China Autom. Congr. (CAC)*, Chongqing, China, Nov. 2023, pp. 4470–4474.
- [22] B. K. Patle, N. Pagar, D. R. K. Parhi, and S. Sanap, "Hybrid FA-GA controller for path planning of mobile robot," in *Proc. Int. Conf. Intell. Controller Comput. Smart Power (ICICCCSP)*, Hyderabad, India, Jul. 2022, pp. 1–6.
- [23] J. Xin, J. Zhong, F. Yang, Y. Cui, and J. Sheng, "An improved genetic algorithm for path-planning of unmanned surface vehicle," *Sensors*, vol. 19, no. 11, p. 2640, Jun. 2019.
- [24] W. Tao, S. Yan, F. Pan, and G. Li, "AUV path planning based on improved genetic algorithm," in *Proc. 5th Int. Conf. Autom., Control Robot. Eng. (CACRE)*, Sep. 2020, pp. 195–199.
- [25] M. Bresciani, F. Ruscio, S. Tani, G. Peralta, A. Timperi, E. Guerrero-Font, F. Bonin-Font, A. Caiti, and R. Costanzi, "Path planning for underwater information gathering based on genetic algorithms and data stochastic models," *J. Mar. Sci. Eng.*, vol. 9, no. 11, p. 1183, Oct. 2021.
- [26] J. Hong, R.-G. Tsai, X. Chen, D. Lin, Y. Yu, Y. Lin, and R. Li, "U: GA-based path planning algorithm for surface floating garbage cleaning robot," *J. Intell. Fuzzy Syst.*, vol. 46, no. 1, pp. 837–850, Jan. 2024.
- [27] J. Hu, M. J. Er, T. Liu, and S. Wang, "A hybrid algorithm based on ant colony and genetic algorithm for AUV path planning," in *Proc. 2nd Conf. Fully Actuated Syst. Theory Appl. (CFASTA)*, Qingdao, China, Jul. 2023, pp. 888–893.
- [28] Y. Wang, W. Li, and R. Jiang, "A novel hybrid algorithm based on improved particle swarm optimization algorithm and genetic algorithm for multi-UAV path planning with time windows," in *Proc. IEEE 5th Adv. Inf. Manage., Communicates, Electron. Autom. Control Conf. (IMCEC)*, vol. 5, Chongqing, China, Dec. 2022, pp. 1005–1009.
- [29] C. Huang, Y. Zhao, M. Zhang, and H. Yang, "APSO: An A-PSO hybrid algorithm for mobile robot path planning," *IEEE Access*, vol. 11, pp. 43238–43256, 2023.
- [30] Z. Du and S. Liu, "Asymptotical RRT-based path planning for mobile robots in dynamic environments," in *Proc. 37th Chin. Control Conf. (CCC)*, Wuhan, China, Jul. 2018, pp. 5281–5286.
- [31] O. Khattab, A. Yasser, M. A. Jaradat, and L. Romdhane, "Intelligent adaptive RRT path planning algorithm for mobile robots," in *Proc. Adv. Sci. Eng. Technol. Int. Conf. (ASET)*, Dubai, United Arab Emirates, Feb. 2023, pp. 1–6.
- [32] D. Lin and J. Zhang, "A reinforcement learning based RRT algorithm with value estimation," in *Proc. 3rd Int. Conf. Big Data, Artif. Intell. Internet Things Eng. (ICBAIE)*, Xi'an, China, Jul. 2022, pp. 619–624.
- [33] S. Shan, J. Shao, H. Zhang, S. Xie, and F. Sun, "Research and validation of self-driving path planning algorithm based on optimized A-artificial potential field method," *IEEE Sensors J.*, vol. 24, no. 15, pp. 24708–24722, Aug. 2024.
- [34] L. Xiangde, Z. Xiang, Z. Yi, L. Hua, and H. Wentao, "Global dynamic path planning algorithm based on harmony search algorithm and artificial potential field method," in *Proc. IEEE 6th Inf. Technol. Mechatronics Eng. Conf. (ITOEC)*, vol. 6, Chongqing, China, Mar. 2022, pp. 346–351.
- [35] C. Wang, X. D. Yang, and H. Li, "Improved q-learning applied to dynamic obstacle avoidance and path planning," *IEEE Access*, vol. 10, pp. 92879–92888, 2022.
- [36] Z. Bai, H. Pang, Z. He, B. Zhao, and T. Wang, "Path planning of autonomous mobile robot in comprehensive unknown environment using deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 22153–22166, Jun. 2024.
- [37] A. Vashisth, J. Rückin, F. Magistri, C. Stachniss, and M. Popović, "Deep reinforcement learning with dynamic graphs for adaptive informative path planning," *IEEE Robot. Autom. Lett.*, vol. 9, no. 9, pp. 7747–7754, Sep. 2024.
- [38] S. P. Beeren, H. Ridderinkhof, and J. T. F. Zimmerman, "An analytical study of chaotic stirring in tidal areas," *Chaos, Solitons Fractals*, vol. 4, no. 6, pp. 1011–1029, Jun. 1994.
- [39] H. Li, T. He, S. Wang, S. Luo, and C. Xu, "YOLOv7-Swin: Urban road trash detection based on improved YOLOv7," in *Proc. 3rd Int. Conf. Electron. Inf. Eng. Comput. (EIECT)*, Shenzhen, China, Nov. 2023, pp. 414–418.
- [40] Y. Wei, J. Li, W. Duan, X. Li, X. Zhang, and Y. Huang, "YOLOv7-UAV: Improved YOLOv7 algorithm for small object detection in UAV image scenarios," in *Proc. Int. Conf. Artif. Intell. Things Syst. (AIoTSys)*, Xi'an, China, Oct. 2023, pp. 64–70.
- [41] A. Panthakkan, W. Mansoor, and H. A. Ahmad, "Accurate UAV-based vehicle detection: The cutting-edge YOLOv7 approach," in *Proc. Int. Symp. Image Signal Process. Anal. (ISPA)*, Rome, Italy, Sep. 2023, pp. 1–4.
- [42] R.-G. Tsai, Y.-Y. Tsai, and P.-H. Tsai, "Automation tool for home fire safety check," *IEEE Sensors Lett.*, vol. 5, no. 12, pp. 1–4, Dec. 2021.
- [43] C.-Y. Wang, A. Bochkovskiy, and H.-Y.-M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Vancouver, BC, Canada, Jun. 2023, pp. 7464–7475.



RONGHAO LI is currently pursuing the B.S. degree in cloud computing with the New Engineering Industry College, Putian University, Fujian, China. He is also a Research Assistant with Prof. Di Lin. He participated in the International Collegiate Programming Contest (ICPC), in 2023. His current research interests include cloud computing, the Internet of Things, and path-planning algorithms.



BINGYING ZHANG is currently pursuing the B.S. degree in Internet of Things engineering with the New Engineering Industry College, Putian University, Fujian, China. She is also a Research Assistant with Prof. Di Lin. Her current research interests include the Internet of Things (IoT), mechatronics integration, and path-planning algorithms.



DI LIN received the B.S. degree in mechanical and electronic engineering from China Jiliang University, China, in 2011. In 2020, he joined the New Engineering Industry College, Putian University, where he currently serves as a Senior Engineer and the Director of the Research Office. His research interests include robotics, computational intelligence, path-planning algorithms, and the IoT.



ZIQI LIN received the B.S. degree in software engineering and cloud computing from the New Engineering Industry College, Putian University, Fujian, China, in 2023. She is currently a Counselor at the School of Information Technology, Xiamen Huatian International Vocation Institute. She participated in the National Internet+ Competition and the National University Internet of Biology Competition. Her research interests include path-planning algorithms, the Internet of Things, and security.



RONG-GUEI TSAI received the M.Eng. degree in computer science from National Chiayi University, Chiayi, Taiwan, in 2010, and the Ph.D. degree from the Institute of Manufacturing Information and Systems, National Cheng Kung University, Tainan, Taiwan, in 2019. In 2024, he joined the School of Computer Science, Guangdong University of Science and Technology, as an Associate Professor. His research interests include sensor networks, the Internet of Things, localization algorithms, and path-planning algorithms.



XINXIN CHEN is currently pursuing the B.S. degree in cloud computing with the New Engineering Industry College, Putian University, Fujian, China. She is also a Research Assistant with Prof. Lin Di. She participated in the National Collegiate Internet of Things Competition and the Internet Plus Competition. Her current research interests include cloud computing, image processing, and path-planning algorithms.



WENWEN ZOU received the B.S. degree in data science and big data technology from the New Engineering Industry College, Putian University, Fujian, China, in 2023. She is currently a Counselor with the School of Information Technology, Xiamen Huatian International Vocation Institute. She participated in the National University Internet of Biology Competition. Her research interests include path-planning algorithms, data visualization, and big data.



SHANNA HE received the B.S. degree in Internet of Things engineering from the New Engineering Industry College, Putian University, Fujian, China, in 2023. She is currently a Counselor at the School of Mechanical and Electrical Engineering and Information, Zhangzhou Institute of Science Technology. She participated in the National Internet+ Competition and the National University Internet of Biology Competition. Her research interests include path planning algorithms, the Internet of Things, and embedded systems development.



YATING YAO is currently pursuing the B.S. degree in innovation, entrepreneurship, and creativity with Fuzhou Technology and Business University, Fujian, China. She participated in the National Collegiate Internet of Things Competition and the Internet Plus Competition. Her current research interests include statistics, image processing, and path-planning algorithms.

...